

1 Lesson 25-09-26

1.1 Introduction to VHDL

Let's start delving into the **VHDL** (*VHSIC Hardware Description Language*) language. There are 3 description styles in VHDL:

- Behavioral:
 - Specifies only the behavior at a higher level of abstraction.
 - Does not imply any particular structure or technology.
- Structural:
 - Specifies more details.
 - Components used and the structure of the interconnection between the components are clearly specified.
 - At a low level of abstraction
- Dataflow (*Register Transfer Language (RTL)*):
 - Data path and control signals are specified.
 - System is described in terms of the data transfer between registers.
 - At an intermediate level of abstraction

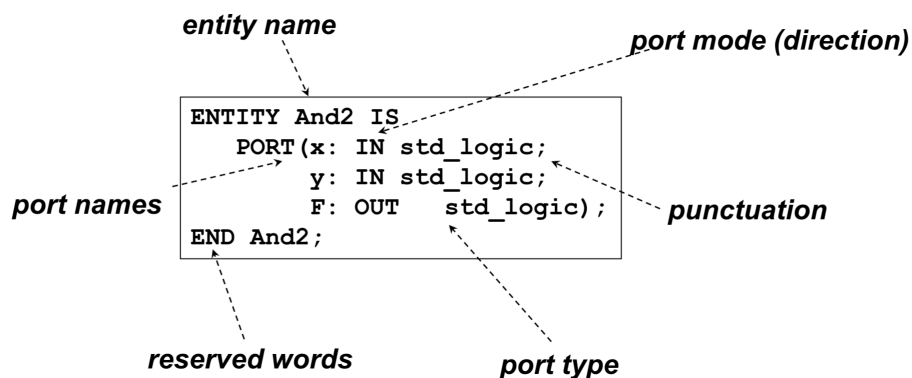
We also note that VHDL's syntax is Ada-based, case insensitive.

1.1.1 VHDL entities

A simple VHDL file contains a **design entity**, comprised of:

- An *entity declaration*;
- An *architecture body*.

Let's look at a very simple entity declaration:



As we know the declaration gives a *black box* view of the component, without its internal architecture.

PORT specifies a list of inputs and outputs of entity.

- `x: IN std_logic`: port `x` is an input port of type `std_logic`;
- `std_logic`: short for "standard logic", a logical bit type (versus say an integer type), which can be '0' or '1' (more later);
- Ports in list separated by semicolons.

A number of port modes are available:

- **IN**: Used for a signal that is an input to an entity.
- **OUT**: Used for a signal that is an output from an entity. The value of the signal can not be used inside the entity. This means that in an assignment statement, the signal can appear only to the left of the `<=` operator.
- **INOUT**: Used for a signal that is both an input to an entity and an output from the entity.
- **BUFFER**: Used for a signal that is an output from an entity. The value of the signal can be used inside the entity, which means that in an assignment statement, the signal can appear both on the left and right sides of the `<=` operator.

BUFFER differs from **INOUT** because of the following:

- It can't have more than 1 source;
- The only signal connected to it is another **BUFFER** port or signal with at most 1 source.

Logic types: VHDL supports different data types for logic. We'll mainly use **standard logic** as defined by IEEE 1164 standard (end of the 1980s).

- Values for simulation: 'U' (unaffected), 'X' (unknown), 'Z' (high impedance), 'W' (weak unknown), 'L' (weak 0), 'H' (weak 1), '-' (don't care), 0, 1
- package `std_logic_1164`: it contains the enumerated types `std_logic` and `std_ulogic`. The main difference between `std_logic` (standard logic) and `std_ulogic` (standard unresolved logic) is:
 - `u` stands for *unresolved* signal: it is illegal to force two values at the same time on a signal of type `std_ulogic`;
 - `std_logic`: there exist resolution functions that define what happens when two values are forced at the same time on a signal.

1.1.2 VHDL architectures

An **architecture** defines an entity's inside — an entity's functionality.

- It describes the behavior or the inner implementation of an entity;
- There may be several architectures for the same entity (for simulation, but not for synthesis);
- Each architecture must be bound to an entity.

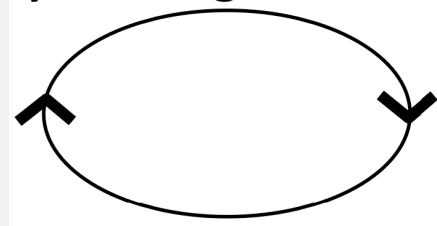
One way to describe architecture is thorough PROCESS: a PROCESS takes in arguments, and reacts to changes of those arguments by producing an output value. By *reactive*, we mean that the process is idle until something happens to the arguments, at which points it *reacts* by *doing something*. Because of this, the argument list is usually called the *sensitivity list*.

Let's take, for instance:

```

1 ENTITY And2 IS
2   PORT (x: IN std_logic;
3         y: IN std_logic;
4         F: OUT std_logic);
5 END And2;
6
7 ARCHITECTURE And2_beh OF And2 IS
8 BEGIN
9
10  PROCESS(x, y) -- this is the process
11  BEGIN
12      F <= x AND y;
13  END PROCESS;
14
15 END And2_beh;
```

*wait until x or
y changes*



$F \leq x \text{ AND } y$

If x and y change in the above example, the process sets the output *F* to the logical AND of x and y. After executing its statements, the process waits until any of its arguments changes again, and then re-executes: a process is thus an infinite loop.

Something we should go through is **concurrent** statements. We know that a typical VHDL architecture is:

```

1 ARCHITECTURE architecture_name OF name_of_entity IS
2   -- Declarations
3   -- components declarations
4   -- signal declarations
5   -- constant declarations
6   -- function declarations
7   -- procedure declarations
8   -- type declarations
9
10  BEGIN
11      -- concurrent statements
12 END architecture_name;
```

The important part of an architecture are its *concurrent statements*. These are statements which can potentially execute at the same time, while interacting. There are a number of concurrent statements in VHDL:

- Concurrent *processes*;
- Concurrent signal assignments;
- Component instantiations;
- Procedure calls;
- Generate statements: these are particularly useful for procedural generation of components, which in hardware are often repeated. This exploits the intrinsic *regularity* of hardware.

We focus on processes.

Processes model hardware's intrinsic concurrency:

- A process is one single concurrent operation;
- Statements within a process are sequential;
- *Concurrency* comes to mean multiple interacting processes.

A process declaration entails signals, variables and constants used in the process. These are to be enclosed between `begin` and `end` keywords. The statements are then sequentially executed.

A process is normally idle:

- If there is a sensitivity list (list of signals to which the process is sensitive), an event on one of the signals triggers the process
- If there is no sensitivity list, the process will always be in execution until a `wait` statement is reached
- Processes may alternatively have a sensitivity list or one or more wait statements (but not both)

Assignment to signals is carried out as follows:

- An assignment to a signal generates a projected value (projected value: value the signal will take later on);
- All signals in a process are all updated at the end of the execution of the process with their projected values;
- In the meantime, if the signal is used for a computation or assignment, the old value is used.

So we have *synchronization* at process level.

Important implications of this are that:

- Process execution order in a cycle doesn't matter;
- The order of assignments to different signals in a process doesn't matter.

One thing to note is that later signal assignment in process has precedence over earlier writ.

Simulation cycle: it can be useful to look at the simulation cycle, that is to consider how an HDL simulator actually works. HDL simulation is complex, so we'll introduce a simplified form. A simulator's job is to *determine values* for signals over time. It does so by repeatedly executing and suspending processes.

Let's look at the example:

```

1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY SimEx1 IS
5     PORT (Q: OUT std_logic );
6 END SimEx1;
7

```

```

8 ARCHITECTURE SimEx1Arch OF SimEx1 IS
9   SIGNAL Clk, S: std_logic;
10
11 BEGIN
12   P1: PROCESS      -- clock generator
13     BEGIN
14       Clk <= '0';
15       WAIT FOR 10 NS;
16       Clk <= '1';
17       WAIT FOR 10 NS;
18     END PROCESS P1;
19
20   P2: PROCESS(S) -- output stage
21     BEGIN
22       Q <= NOT(S);
23     END PROCESS P2;
24
25   P3: PROCESS      -- clock divider
26     BEGIN
27       -- legacy syntax for rising edges
28       WAIT UNTIL Clk='1' AND Clk'EVENT;
29       S <= '1';
30       WAIT UNTIL Clk='1' AND Clk'EVENT;
31       S <= '0';
32       WAIT;
33     END PROCESS P3;
34
35 END SimEx1Arch;

```

Time (ns):	Init	0	10	10	10	20	30	30	30	40	50
Signals											
Q	'U'	'U'	'U'	'U'	'0'	'0'	'0'	'0'	'1'	'1'	'1'
Clk	'U'	'0'	'1'	'1'	'1'	'0'	'1'	'1'	'1'	'0'	'1'
S	'U'	'U'	'U'	'1'	'1'	'1'	'1'	'0'	'0'	'0'	'0'

Concurrent signal assignments can be seen as a sugared version of processes with a single signal assignment. The sensitivity list is automatically computed from the assignments right hand side arguments. They are used in dataflow descriptions for:

- Descriptions of combinational logic;
- Interconnection of components at a lower hierarchical level.

For instance, let's look at the example:

```

1 ARCHITECTURE Datafl OF fulladder IS
2 BEGIN
3   s <= (a XOR b) XOR ci;
4   co <= (b AND ci) OR (a AND ci) OR (a AND b);
5 END Datafl;
6
7 -- translates to:
8
9 ARCHITECTURE Behav OF fulladder IS
10 BEGIN
11 PROCESS(a, b, ci)
12 BEGIN

```

```

13 s <= (a XOR b) XOR ci;
14 co <= (b AND c) OR (a AND c) OR (a AND b);
15 END PROCESS;
16 END Behav;

```

1.1.3 VHDL testbenches

Once we define an entity, we want to study how our new entity behaves. We could do this manually, but we have software tools exactly for this reason. We perform a **simulation**: User provides input values, simulator generates output values.

- A simulation takes in **test vectors**, that is a sequence of input values;
- The result is a **waveform**, a graphical depiction of the sequence of values the simulation emits.

To do this we create new **testbench** entity that provides test vectors to component's inputs. An HDL testbench is:

- An entity with no ports;
- Where we declare component to test;
- Declare signal for each port;
- Instantiate component, map signals to ports;
- Process to set signal values at desired times.

An example of a testbench can be:

```

1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY Testbench IS
5 END Testbench;
6
7 ARCHITECTURE TBarch OF Testbench IS
8
9     COMPONENT And2 IS
10         PORT (
11             x : IN  std_logic;
12             y : IN  std_logic;
13             F : OUT std_logic
14         );
15     END COMPONENT;
16
17     SIGNAL x_s, y_s, F_s : std_logic;
18
19 BEGIN
20
21     CompToTest : And2
22         PORT MAP (x_s, y_s, F_s);
23
24     PROCESS
25     BEGIN
26         -- Test all possible input combinations
27         y_s <= '0';
28         x_s <= '0';

```

```

29     WAIT FOR 10 ns;
30
31     y_s <= '0';
32     x_s <= '1';
33     WAIT FOR 10 ns;
34
35     y_s <= '1';
36     x_s <= '0';
37     WAIT FOR 10 ns;
38
39     y_s <= '1';
40     x_s <= '1';
41     WAIT;
42 END PROCESS;
43
44 END TBarch;

```

1.1.4 Combinatorial circuits

A **circuit**, in the broadest sense, is connection of components. It's also known as a *structure*. A circuit is a second way to describe an architecture, in contrast to using a process, as earlier.

When used in a circuit, an entity is known as a COMPONENT. An *instance* is an occurrence of a component in a circuit. There may be multiple instances of a component.

This leads us to the third type of concurrent statements: **component instantiations**. Let's look at an example:

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY BeltWarn IS -- root entity
5      PORT (
6          k : IN    std_logic;
7          p : IN    std_logic;
8          s : IN    std_logic;
9          w : OUT   std_logic
10     );
11 END BeltWarn;
12
13 ARCHITECTURE Circuit OF BeltWarn IS -- root structural description
14
15     COMPONENT And2 IS
16         PORT (
17             x : IN    std_logic;
18             y : IN    std_logic;
19             F : OUT   std_logic
20         );
21     END COMPONENT;
22
23     COMPONENT Inv IS
24         PORT (
25             x : IN    std_logic;
26             F : OUT   std_logic
27         );
28     END COMPONENT;
29
30     SIGNAL n1, n2 : std_logic; -- signal declaration
31
32 BEGIN -- behavoir starts here

```

```

33
34   And2_1 : And2 PORT MAP (k, p, n1);
35   Inv_1   : Inv   PORT MAP (s, n2);
36   And2_2 : And2 PORT MAP (n1, n2, w);
37
38 END Circuit;

```

When we create a circuit we have to:

- Declare a new entity;
- Declare its components.

We then want to connect components together. **Signal** declaration statements can be useful for this. A SIGNAL is like a variable in a traditional programming languages: it stores value, but at particular time. Signals may be explicitly declared, but an entity's ports are also signals.

We use signals in *port maps*.

Port maps are parts of *component instantiations* that map the ports of a component to other ports of the root entity, or signals defined within that entity.

At this point some more things about *testbenches* should be clear:

- Note use of component instantiation statements;
- Signals are used because process cannot access instantiated component's ports directly;
- Component instantiation statements and process statements are concurrent statements and they can both appear in an architecture.

1.2 Top-down design

Let's introduce the **top-down** design philosophy. Designer may initially know system behavior, but not structure.

The idea of top-down design is to:

- Capture behavior, and simulate;
- Capture structure (circuit), simulate again;
- Gets behavior right first, unfettered by complexity of creating structure.

How can we describe behavior? One way in VHDL is to use a *process*. By making a process sensitive to its inputs, the process executes if a change occurs on any of those inputs. The simplest process uses one signal assignment statement.

We can then simulate the behavior captured by our process using testbench (same as shown earlier) to get waveforms. We can, this way, proceed to capture structure, simulate again using same testbench. The result should of course be the same waveforms.

It is useful to have signal assignment statements, as they assign values to signals which can be any combination of different operators. Note: A port is also a signal. Lastly, a process may have multiple signal assignment statements.

We will now cover a few more constructs which can be useful to capture high level behavior.

If statements are supported by VHDL. The condition *has* to be boolean, and errors will be reported at compile time.

We see an example:

```

1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3 ENTITY BeltWarn IS
4   PORT (
5     k, p, s : IN std_logic;
6     w : OUT std_logic
7   );
8 END BeltWarn;
9 ARCHITECTURE Behav OF BeltWarn IS
10 BEGIN
11   PROCESS (k, p, s)
12   BEGIN
13     IF ((k AND p AND NOT(s)) = '1') THEN
14       w <= '1';
15     ELSE
16       w <= '0';
17     END IF;
18   END PROCESS;
19 END Behav;

```

ELSIF is also supported, in Python fashion.

Vectors are provided by `std_logic_vector`. These represent collection of bits, which is more convenient than declaring separate bits like `d3`, `d2`, `d1`, `d0`. A nice feature is that a vector's bits are numbered: the ordering is specified in the syntax: `(0 TO 3)`, `(1 TO 4)`, ... Assignment is possible with strings of bits.

An example is:

```

1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY Dcd2x4vect IS
5   PORT (i: IN std_logic_vector(0 TO 1);
6         d: OUT std_logic_vector(0 TO 3);
7   END Dcd2x4vect

```

Switch statements are also available, and they are called CASE statements. These are preferred over IF-THEN-ELSE when a single expression determines which statement(s) to execute. The selection keyword is `WHEN`, and the sequence of statements after a `WHEN` includes all statements between the `WHEN` itself and the next `WHEN` or `END CASE`.

An example is:

```

1 ARCHITECTURE Dcd2x4_vect OF Dcd2x4vect IS
2 BEGIN
3   PROCESS (i)
4   BEGIN
5     CASE i IS
6       WHEN "00" => d <= "0001";
7       WHEN "01" => d <= "0010";
8       WHEN "10" => d <= "0100";
9       WHEN OTHERS => d <= "1000";
10    END CASE;
11  END PROCESS;
12 END Dcd2x4_vect

```

The *case sequential* statement is allowed, that is a `WHEN` clause can include more than one choice:

```

1 CASE expression IS
2   WHEN choices =>
3     -- sequential statements
4   WHEN choices =>
5     -- sequential statements
6     -- (branches are allowed)
7   WHEN OTHERS =>
8     -- sequential statements
9 END CASE;

```

1.3 Design units

VHDL allows for **design units**. They are VHDL code segments that can be compiled separately and stored in a library for future use. Libraries are collections of compiled design units that are mapped to physical directories. A library is a set of components (entities and architectures) and packages.

There are 5 types of design units:

1. Entity;
2. Architecture;
3. Package;
4. Package body;
5. Configuration.

1.3.1 Packages

Packages are repositories for type definitions, procedures and functions. They group common declarations used by several design units. Standard packages are defined in generic libraries (e.g. IEEE) for numeric/text handling (for example, `std_logic_1164`, `numeric_std`, `std_logic_arith`, `std_logic_signed`, `std_logic_unsigned`, `TEXTIO`, ...).

A package consists of:

- A package **declaration** (with declarations of types, subtypes, constants, global signals, functions and procedures, attributes, files, components, ...).

Because a package serves as a central repository for frequently used utilities, such as component declarations, ... the declarations may then be reused by any VHDL model by simply accessing the package.

Usually, to use a package we use the `USE` clause.

An example of a package declaration is:

```

1 PACKAGE example IS
2   TYPE winter IS (Dec, Jan, Feb);
3   COMPONENT And3 IS
4     PORT (
5       x : IN std_logic;
6       y : IN std_logic;
7       z : IN std_logic;
8       F : OUT std_logic
9     );
10  END COMPONENT;
11  CONSTANT pin2pin_delay : TIME := 125ns;

```

```

12 FUNCTION int2bit_vect (int_val : INTEGER) RETURN bit_vector;
13 END example;

```

- A package **body**: it contains the implementations of functions, procedures, ... A package body is always associated with a package declaration.

An example of a package body is:

```

1 PACKAGE BODY example IS
2   FUNCTION int2bit_vect (int_val : INTEGER) RETURN bit_vector IS
3     BEGIN
4       -- behavior of function described here
5     END int2bit_vect;
6 END example;

```

1.3.2 Configurations

VHDL description may consist of many design entities, each with several architectures, and organized into a design hierarchy. A **configuration** does the job of specifying the exact set of entities and architectures used in a particular simulation or synthesis run.

A configuration does two things:

- It specifies the design entity used in place of each component instance (i.e., it plugs the chip into the chip socket and then the socket-chip assembly into the PCB);
- It specifies the architecture to be used for each design entity (i.e., which die).

A configuration statement is used to bind a component instance to an entity-architecture pair. A configuration can be considered as a parts list for a design. It describes which behavior to use for each entity, much like a parts list describes which part to use for each part in the design.

Component configuration can be performed outside the architecture body which instantiates a certain component. A configuration declaration is a design unit which can be compiled separately. The particular architecture body has not to be recompiled when the binding is changed.

1.3.3 Libraries

Libraries are collections of compiled design units that are mapped to physical directories. A library is a set of components (entities and architectures) and packages.

There are two reserved library names always available to designers:

- work: predefined library;
- STD: it contains 2 packages:
 - STANDARD provides declaration for predefined types (real, integers, Boolean, etc);
 - TEXTIO contains useful subprograms that enables to perform ASCII file manipulation.

The statement `library STD;` declares STD to be a library. The statement `use STD.STANDARD.ALL;` allows the use of all declarations in package STANDARD located within the library STD. The statement `library ieee;` declares ieee to be a library. work is the predefined library.